

Propuesta de pipeline de MLOps para el diagnóstico de enfermedades
asistido por Machine Learning

Autores:

Andrés Felipe Cano Larrahondo

Jhonattan Rafael Reales De La Asunción

Universidad ICESI

Maestría en Inteligencia Artificial Aplicada
Semestre II

Curso: Machine Learning Operations

22 de Noviembre de 2025

DESARROLLO DE LA PROPUESTA

1. Introducción y contexto del problema

En el contexto actual de la medicina, la cantidad de información disponible sobre los pacientes ha crecido de forma exponencial. Historias clínicas electrónicas (EHR), resultados de laboratorio, imágenes médicas, notas clínicas y señales fisiológicas se combinan para formar un ecosistema de datos rico, pero altamente heterogéneo. Sin embargo, para ciertas patologías poco frecuentes —conocidas como enfermedades huérfanas— la cantidad de datos disponibles sigue siendo muy limitada, lo que dificulta la creación de modelos robustos únicamente a partir de estas muestras.

El objetivo de este trabajo es proponer un pipeline de MLOps end-to-end que permita desarrollar, desplegar y operar un sistema inteligente capaz de predecir, a partir de datos clínicos de un paciente, la probabilidad de que padezca una enfermedad, distinguiendo entre al menos cinco niveles de severidad:

- **No Enfermo**
- **Enfermedad Leve**
- **Enfermedad Aguda**
- **Enfermedad Crónica**
- **Enfermedad Terminal**

Este sistema debe ser capaz de manejar y pronosticar tanto enfermedades comunes (muchos datos) como enfermedades huérfanas (pocos datos), cuidando aspectos críticos como:

- Privacidad y seguridad de la información clínica.
- Manejo del desbalance de clases y la escasez de datos.
- Explicabilidad mínima para soporte a la decisión clínica.
- Capacidad de operar de forma confiable en producción, con monitoreo y reentrenamiento continuo.

A diferencia de un enfoque puramente académico de Machine Learning, en este documento se plantea una solución desde la perspectiva de MLOps, considerando el ciclo de vida completo del modelo: desde la ingesta y versionamiento de datos, pasando por el entrenamiento y validación, hasta la entrega continua (CI/CD), el monitoreo del modelo en producción y los procesos de retraining.

1.1 Usuarios finales y casos de uso

El principal usuario final del sistema es el médico especialista (o un grupo de médicos en un hospital) que desea apoyar su criterio clínico con una herramienta de soporte a la decisión. A partir de los síntomas, variables clínicas y otros datos del paciente, el sistema sugiere una probabilidad o clase de enfermedad, acompañada de explicaciones de alto nivel sobre los factores que influyeron en la predicción.

Casos de uso principales:

- Apoyo al diagnóstico inicial: el médico introduce información del paciente (variables tabulares, resultados de laboratorio, etc.) y recibe una predicción de riesgo asociada a determinadas enfermedades.
- Seguimiento de pacientes con enfermedades crónicas: el modelo se utiliza periódicamente para evaluar la evolución del paciente y detectar cambios relevantes.
- Detección temprana de posibles enfermedades huérfanas: a partir de patrones de síntomas atípicos, el sistema puede sugerir la posibilidad de una enfermedad poco frecuente que merezca un estudio más profundo.

El sistema no reemplaza la decisión médica, sino que actúa como un sistema de soporte a la decisión clínica.

1.2 Requerimientos no funcionales

Para que este sistema sea útil en un entorno real, se deben cumplir varios requerimientos no funcionales clave:

- **Privacidad y cumplimiento normativo:** manejo estricto de datos sensibles, siguiendo marcos como HIPAA/GDPR y las regulaciones locales.
- **Baja latencia:** las predicciones deben obtenerse en tiempos adecuados para un contexto clínico (segundos, no minutos).
- **Disponibilidad y confiabilidad:** el servicio debe ser altamente disponible y resiliente a fallos en producción.
- **Escalabilidad:** el sistema debe poder manejar un aumento en el volumen de solicitudes, especialmente en enfermedades comunes.
- **Auditabilidad y trazabilidad:** cada predicción debe poder rastrearse al modelo, versión y datos que la generaron.
- **Explicabilidad mínima:** el modelo debe proveer, al menos, explicaciones globales y/o locales (importancia de variables, contribuciones por feature) que permitan aumentar la confianza del médico.

1.3 Visión MLOps del problema

Desde una perspectiva de MLOps, este problema no se limita a “entrenar un modelo de clasificación médica”, sino a diseñar un ecosistema completo que:

- Permita iterar rápidamente sobre nuevas versiones de modelos.
- Automatice las pruebas, empaquetado y despliegue (CI/CD).
- Monitoree tanto el comportamiento del sistema como los datos en producción para detectar drift y degradación del modelo.
- Habilite procesos de reentrenamiento continuo o periódico, aprovechando nuevos datos clínicos que se vayan registrando.
- Soporte diferentes escenarios de consumo:
 - Ejecución local (modelo ligero) en el computador del médico.
 - Consumo vía API desde la nube (modelo desplegado como microservicio).

La propuesta de pipeline que se presenta a continuación integra las distintas etapas del ciclo de vida del modelo (datos, entrenamiento, despliegue, monitoreo y retraining) con tecnologías concretas, siguiendo los principios estudiados en las unidades del curso.

2. Supuestos y Restricciones del Sistema

Esta sección deja explícito el marco en el que opera el pipeline end-to-end propuesto. Cada suposición tiene implicaciones directas en diseño, infraestructura, validación y despliegue.

2.1 Supuestos del dominio clínico

Dado que el problema original es abierto, se asume que:

S1. Los datos provienen de hospitales con EHR estructurados

- Los datos tabulares (edad, sexo, signos vitales, exámenes) vienen de sistemas HL7/FHIR.
- Los datos de imágenes provienen de PACS (radiografías, TAC, RM, etc.).
- Las notas médicas están disponibles como texto clínico no estructurado.

Implicación: Se necesitan pipelines distintos para cada tipo de dato, lo cual justifica el uso de un *Feature Store* y un *Data Validation Layer*.

S2. Existe un equipo clínico responsable de generar y validar las etiquetas diagnósticas

Las etiquetas de tipo y severidad de la enfermedad —incluyendo tanto patologías comunes como enfermedades raras— provienen de documentación clínica revisada, confirmadas por un equipo médico especializado (médicos tratantes, especialistas y/o comité clínico).

Implicación: El pipeline debe incluir un proceso robusto de label quality management, apoyado en herramientas como AWS SageMaker Ground Truth. Este componente permitirá implementar validación clínica, trazabilidad completa de las etiquetas, versionado del dataset, detección de inconsistencias y flujos de auditoría previos al entrenamiento.

S3. El sistema será usado como soporte a la decisión, no como diagnóstico autónomo

- El modelo complementa la visión del médico, no la reemplaza.
- El usuario final es un profesional de la salud.

Implicación: Se requiere explicabilidad mínima (SHAP/LIME/interpretaciones locales) y transparencia de predicciones.

S4. No se puede usar información que identifique directamente al paciente

(HIPAA / GDPR)

Implicación:

- Se aplican procesos de anonimización, pseudonimización y control de acceso.
- No se almacenan nombres, direcciones, ni números de historia clínica en el pipeline.

2.2 Supuestos técnicos

S5. Los datos tienen suficiente calidad para entrenamiento

Aunque existe ruido, se asume que las variables clave pueden ser limpiadas y validadas con reglas claras (rangos fisiológicos, tipos de datos, outliers médicos).

Implicación: Se debería usar una herramienta o servicio que sirva como un validador de calidad de datos antes de entrenar o realizar inferencias (e.g. AWS Glue Data Quality, Great expectations + SageMaker)

S6. El sistema de predicción debe poder ejecutarse tanto en infraestructura local como en la nube

El modelo de predicción debe soportar un **esquema de despliegue híbrido**, para adaptarse a distintos escenarios operativos dentro del sistema de salud:

- **Modo local (on-device):** versión ligera del modelo (ONNX) para uso en laptop del médico. Esto permite funcionamiento offline, baja latencia y uso en entornos donde la conectividad es limitada o está regulada por políticas hospitalarias.
- **Modo cloud:** Servicio centralizado, expuesto vía API segura, alojado en una nube o servidor institucional. Modalidad adecuada para hospitales grandes que requieren mayores recursos de interpretabilidad de las predicciones, facilitar el registro centralizado, auditoria e integraciones con otros sistemas de información en línea.

Implicación: El sistema de predicción debe ser portátil y reproducible en ambos entornos. Lo anterior se puede lograr usando contenedores Docker. Además, se necesita un pipeline de CI/CD transversal que genere y versione automáticamente las imágenes para ambos entornos, asegurando consistencia entre ejecuciones.

S7. El modelo debe ejecutarse con baja latencia

El modelo debe ofrecer baja latencia en las predicciones, con tiempos objetivo <3s tanto en modo local como cloud.

Implicación: Se deben priorizar modelos optimizados y exportables a ONNX, incorporando técnicas opcionales como cuantización o reducción de precisión, y empleando runtimes acelerados para mantener latencia consistente en entornos locales y cloud.

S8. La infraestructura cuenta con un proceso de CI/CD establecido

La infraestructura dispone de CI/CD basado en GitHub Actions, Docker y un proveedor cloud (AWS). La implementación de CI/CD garantiza que cada cambio en el código, los modelos y los contenedores se valide automáticamente, reduciendo errores, acelerando los ciclos de entrega y asegurando despliegues consistentes y reproducibles. Esto es especialmente importante en entornos clínicos, donde la confiabilidad del sistema y la trazabilidad de cada versión son críticas.

Implicación: El pipeline integrará pruebas automáticas, construcción y versionado de contenedores, publicación en un registry seguro y despliegue automatizado con controles de seguridad y validación.

3. Arquitectura MLOps y Flujos Operacionales del Sistema

La arquitectura propuesta se basa en una visión de MLOps end-to-end sobre AWS, organizada en dos grandes macroflujos:

- **Flujo offline (batch):** ingesta y preparación de datos, etiquetado clínico, entrenamiento, evaluación y registro de modelos.
- **Flujo online (tiempo casi real):** servicio de inferencia para consumo por parte de los médicos, tanto en la nube como en entornos locales.

Sobre estos dos macroflujos se despliegan, de forma más detallada, cuatro flujos lógicos:

- Pipeline de datos y etiquetado clínico
- Pipeline de entrenamiento y registro de modelos.
- Pipeline de despliegue e inferencia (cloud y local).
- Monitoreo, detección de drift y reentrenamiento.

Adicionalmente, existe un flujo transversal de CI/CD que orquesta la construcción, prueba y despliegue de contenedores y modelos en los diferentes entornos (desarrollo, pruebas y producción).

A continuación, definimos con mayor detalle las características de cada flujo lógico del sistema.

3.1 Pipeline de datos y etiquetado clínico

Este flujo se encarga de llevar los datos clínicos desde las fuentes hospitalarias hasta un estado listo para entrenamiento y para inferencia.

- **Fuentes de datos clínicas:**
EHR tabular estructurado (HL7/FHIR), imágenes médicas (PACS, DICOM), texto clínico no estructurado (notas médicas) y datos históricos de pacientes
- **Ingesta y almacenamiento inicial (Data Lake):**
Los datos crudos se almacenan en **Amazon S3** en “buckets” diferenciados por tipo de dato y entorno (raw, processed, features). Opcionalmente se apoya en **AWS Glue** para catalogar las tablas y esquemas (Glue Data Catalog) y en **AWS Lake Formation** para control de acceso y seguridad del data lake.
- **Calidad y validación de datos:**
Se aplican reglas de validación de esquema y rangos fisiológicos usando el framework **Great Expectations** integrados en **SageMaker Processing Jobs**.
- **Etiquetado clínico y gestión de labels:**
Para la construcción de datasets etiquetados y validación de etiquetas por parte del equipo médico se utiliza **Amazon SageMaker Ground Truth** u otra solución de etiquetado acorde a los equipos clínicos. Se mantiene trazabilidad completa de quién etiquetó qué, cuándo y bajo qué versión de guía clínica.
- **Feature Store:**
Las variables derivadas (features tabulares, embeddings de texto, features de imágenes) se almacenan en un **Feature Store centralizado (Amazon SageMaker Feature Store)**, separando entidades (paciente, episodio, evento) y manteniendo versiones de features.

Este flujo alimenta directamente al **pipeline de entrenamiento** y garantiza que los datos utilizados en inferencia sigan las mismas transformaciones que en entrenamiento.

Los flujos de ingesta y transformación se implementan en **Python**, utilizando **PySpark** para los procesos batch de mayor volumen, aprovechando el procesamiento distribuido sobre EMR/Glue y la integración nativa con S3.

3.2 Pipeline de entrenamiento y registro de modelos (offline)

El pipeline de entrenamiento se implementa en **Python** (notebooks y scripts), orquestado por **Amazon SageMaker Pipelines** y **Airflow/MWAA**, y puede apoyarse en **PySpark** para la preparación distribuida de datasets de entrenamiento de gran tamaño. Los pasos principales son:

- **Extracción de features de entrenamiento:**
Lectura del Feature Store y de S3 para construir el dataset de entrenamiento y validación, respetando ventanas temporales (para evitar leakage) y reglas clínicas.
- **Manejo de desbalance y pocas muestras:**
Aplicación de técnicas como *oversampling*, *class weights*, *focal loss* o *meta-learning* para manejar el desbalance entre enfermedades comunes y enfermedades huérfanas.
- **Entrenamiento de modelos multimodales (tipo stacking):**
Entrenamientos separados para:
 - Datos tabulares (XGBoost, LightGBM, redes neuronales tabulares, etc.).
 - Texto clínico (modelos tipo BERT clínico, embeddings preentrenados).
 - Imágenes médicas (CNN, Transfer learning y/o modelos preentrenados).Luego se aplica una integración multimodal, con una fusión de predicciones de la data tabular + features de textos + features de imagenes.
- **Búsqueda y optimización de hiperparámetros:**
Uso de **SageMaker Hyperparameter Tuning Jobs** para explorar configuraciones y optimizar métricas clínicas clave (sensibilidad, especificidad, AUROC, etc.).
- **Evaluación y métricas clínicas:**
 - Generación de métricas por nivel de severidad (No enfermo, Leve, Aguda, Crónica, Terminal) y análisis de errores por subgrupos de pacientes.
 - Generación de explicaciones globales y locales (SHAP/LIME) como parte de los artefactos del experimento.
- **Registro y versionado de modelos:**
 - Los modelos entrenados, junto con sus métricas, parámetros y artefactos, se registran en el **SageMaker Model Registry**.
 - Los artefactos (pesos, configuraciones, gráficos de métricas, explicaciones) se almacenan en **S3** con versionado habilitado.
 - Para producción solo se promueven los modelos que superan umbrales clínicos acordados con el equipo médico.

3.3 Pipeline de despliegue e inferencia (cloud y local)

Una vez registrado, el modelo se empaqueta y se expone en dos modalidades:

1. Modo cloud (microservicio de inferencia):

El modelo se empaqueta en una imagen **Docker** almacenada en **Amazon ECR**. El servicio de predicción se despliega como endpoint en un microservicio **FastAPI** corriendo sobre **Amazon ECS/Fargate**. El endpoint se expone de forma segura mediante **Amazon API Gateway**, con autenticación (p. ej. JWT, OAuth2) e integración con **AWS IAM**. La aplicación React usada por los médicos puede alojarse en **S3 + CloudFront** o **AWS Amplify**, consumiendo la API de inferencia.

2. Modo local (on-device / on-premise):

El modelo se exporta a **ONNX** como formato estándar para ejecución local. Se genera un contenedor Docker “ligero” con el runtime ONNX, el modelo convertido y un servidor FastAPI local. Este contenedor puede ejecutarse en servidores hospitalarios o incluso en el laptop del médico, sin necesidad de conexión a internet (solo dentro de la red hospitalaria).

En ambos casos, el flujo de inferencia incluye:

- Recepción de datos clínicos del paciente.
- Validación rápida de esquema/rangos en la entrada.
- Consulta al Feature Store cuando corresponda.
- Inferencia del modelo y generación de probabilidades por nivel de severidad y por tipo de enfermedad.
- Inferencia de explicaciones de alto nivel (feature importance local).
- Registro de logs de predicción y metadatos para auditoría.

3.4 Monitoreo, detección de drift y reentrenamiento

El cuarto flujo se encarga de asegurar que el sistema se mantenga clínicamente útil y estable en el tiempo.

- **Monitoreo de datos y modelo:**

Uso de **Amazon CloudWatch** y **SageMaker Model Monitor** (u otras herramientas) para detectar cambios en la distribución de las variables de entrada (data drift), detectar cambios en la distribución de las predicciones (prediction drift) y monitorear latencia, tasa de errores y disponibilidad del endpoint.

- **Alertas y dashboards:**

Definición de umbrales y alarmas (CloudWatch Alarms) que notifiquen al equipo de datos y al equipo clínico cuando:

- Se detecta drift significativo.
- La latencia supera el objetivo (<3s).
- El endpoint presenta errores anómalos.

- **Reentrenamiento automático o programado:**

Cuando se acumulan nuevos datos etiquetados o se detecta drift, se dispara un **nuevo pipeline de entrenamiento (SageMaker Pipelines)** usando los datos más recientes. El nuevo modelo se compara sistemáticamente con el modelo en producción. Solo se promueve si mejora las métricas clínicas o corrige degradaciones. El proceso incluye siempre una etapa de revisión/aprobación clínica antes de activar el nuevo modelo en producción.

3.5 CI/CD transversal (código, contenedores y modelos)

Finalmente, todos los flujos anteriores se integran con un proceso de **CI/CD ya establecido**, basado en:

- **Repositorio GitHub** con el código del proyecto (pipeline de datos, entrenamiento, infra como código, API, frontend).
- **GitHub Actions** como motor de CI/CD que:
 - Ejecuta pruebas unitarias y de integración.
 - Valida el pipeline de datos (tests de esquema y transformaciones).
 - Construye y publica imágenes Docker en **Amazon ECR**.
 - Despliega automáticamente en entornos de dev/stage/prod (SageMaker, ECS/EKS, API Gateway, etc.) usando plantillas de infraestructura como código (**CloudFormation**).
- Controles de seguridad, revisiones de código y aprobación manual para cambios sensibles (por ejemplo, despliegue a producción de un nuevo modelo clínico).

3.6 Orquestación de pipelines (Airflow/MWAA)

Para orquestar los flujos batch de datos, entrenamiento, monitoreo y reentrenamiento, se propone el uso de un orquestador de workflows como Apache Airflow, desplegado en su versión gestionada (AWS Managed Workflows for Apache Airflow, MWAA).

Airflow coordina la ejecución de tareas como:

- Extracción de datos clínicos desde los sistemas hospitalarios y carga en S3.
- Ejecución de jobs de validación de calidad de datos (Glue, Great Expectations).
- Disparo de pipelines de entrenamiento en Amazon SageMaker.
- Evaluación automática de métricas, registro de modelos y notificaciones al equipo clínico.
- Ejecución periódica de tareas de monitoreo de drift y reentrenamiento.

3.7 Seguridad, Cumplimiento y Gobierno del Sistema

El sistema debe ser **seguro, auditado, trazable y alineado con estándares internacionales**, con el fin de reducir los riesgos legales y fortalecer la confianza del personal médico. Debido a lo anterior, proponemos incorporar un marco robusto de seguridad, cumplimiento normativo y gobierno de datos, indispensable para operar con información clínica altamente sensible.

Los principales componentes de seguridad propuestos son:

- **Control de identidades y permisos (IAM):**

El acceso a todos los recursos (**S3, Feature Store, SageMaker, ECS/Fargate, API Gateway y repositorios ECR**) se gestiona mediante políticas de AWS Identity and Access Management (IAM).

- **Cifrado end-to-end (AWS KMS)**

Todos los datos clínicos, tanto en reposo como en tránsito, se cifran utilizando claves administradas en **AWS Key Management Service (KMS)**.

- **Aislamiento de red (VPC + subnets seguras)**

Los servicios críticos se alojan dentro de una **VPC privada**, con subredes públicas mínimas solo cuando sea estrictamente necesario.

- **Comunicación segura (HTTPS/TLS)**

Toda comunicación entre frontend, API Gateway y microservicios utiliza **HTTPS con TLS 1.2 o superior**.

Las solicitudes al endpoint de predicción se firman o validan mediante JWT/OAuth2 o IAM SigV4 según el tipo de cliente (frontend médico vs servicios internos).

- **Auditoría, logging y trazabilidad (CloudTrail, CloudWatch)**

La plataforma activa:

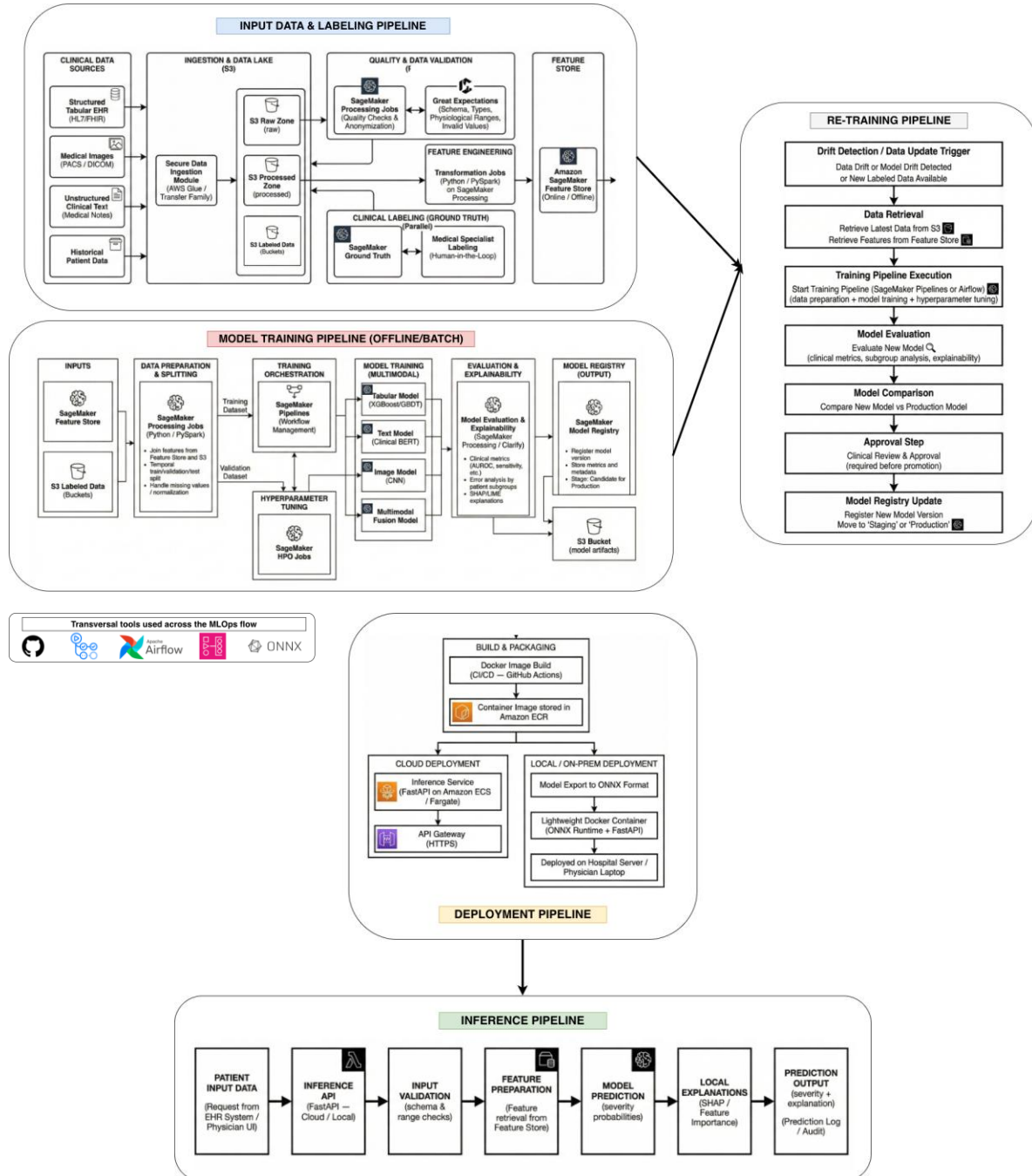
- **AWS CloudTrail** para auditoría detallada de accesos, configuraciones, cambios y uso de servicios.
- **CloudWatch** para logs de inferencia, drift, latencia, fallas, métricas de endpoint.
- **S3 versionado** para trazabilidad de datasets, modelos y artefactos.

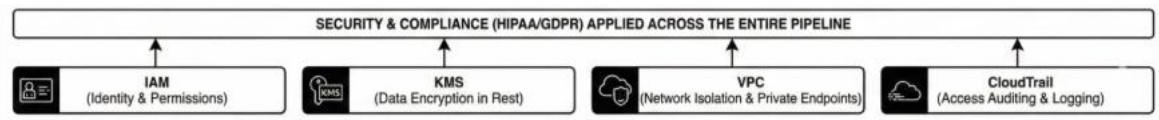
Esto permite reconstruir cualquier evento operacional o clínico (accountability y forensics).

3.8 Justificación de servicios y herramientas usadas

Servicio / Herramienta	Rol en el pipeline	Justificación
S3	Data Lake (raw / processed / features) y repositorio de artefactos	Almacenamiento durable, barato y estándar para ML en AWS.
SageMaker Processing + Great Expectations	Validación y limpieza de datos clínicos	Permite jobs reproducibles y reglas de calidad formales sobre datos sensibles.
SageMaker Ground Truth	Etiquetado clínico supervisado por médicos	Facilita flujos de labeling con trazabilidad y control de calidad de las etiquetas.
SageMaker Feature Store	Gestión centralizada de features para training e inferencia	Garantiza consistencia de features entre entrenamiento y producción, con versionamiento.
SageMaker Pipelines	Orquestación del pipeline de entrenamiento y reentrenamiento	Pipelines declarativos, reproducibles y auditables nativos para ML en AWS.
SageMaker Model Registry	Registro y versionado de modelos	Asegura trazabilidad de modelos, métricas y artefactos, y facilita promoción a producción.
ECR (Elastic Container Registry)	Almacenamiento de imágenes Docker del modelo y servicios	Integración nativa con ECS/Fargate y SageMaker; simplifica gestión de contenedores.
ECS/Fargate	Ejecución del microservicio de inferencia en la nube	Escalabilidad automática sin gestionar servidores (serverless), ideal para workloads clínicos variables.
API Gateway + HTTPS/TLS	Exposición segura del endpoint de predicción hacia las apps médicas	Control de acceso, throttling y terminación TLS para proteger la API clínica.
Frontend React + S3/CloudFront	Interfaz web para que el médico consuma las predicciones	Entrega rápida y segura de la UI con baja latencia y alta disponibilidad.
ONNX Runtime + Docker local	Inferencia en entornos on-premise / sin conexión	Permite ejecutar el mismo modelo en hospitales o laptops, manteniendo portabilidad y desempeño.
MWAA (Airflow gestionado)	Orquestación de flujos batch (datos, training, monitoreo, drift)	DAGs claros, reprogramables y con monitoreo nativo para tareas recurrentes.
CloudWatch + Model Monitor	Monitoreo de métricas, logs e indicadores de drift	Visibilidad centralizada de desempeño, latencia y salud del modelo en producción.
CloudTrail	Auditoría de accesos y cambios en recursos	Soporta cumplimiento regulatorio (HIPAA/GDPR) con trazabilidad de quién hizo qué y cuándo.
IAM + VPC + KMS	Seguridad (identidades, red y cifrado)	Obligatorio para datos clínicos: least privilege, aislamiento de red y cifrado end-to-end.
GitHub + GitHub Actions	Gestión de código y CI/CD de pipelines, contenedores y despliegues	Integración directa con el repositorio, pruebas automáticas y despliegue controlado a AWS.

3.9 Diagrama de la arquitectura





4. Conclusiones

El pipeline propuesto consolida una arquitectura de MLOps **robusta, escalable y segura**, diseñada específicamente para el contexto clínico, donde la calidad del dato, la explicabilidad y la trazabilidad son esenciales. A lo largo del diseño se integran componentes que permiten gestionar de manera confiable todo el ciclo de vida del modelo: desde la ingesta de datos clínicos, pasando por su validación, etiquetado y entrenamiento multimodal, hasta el despliegue, monitoreo y reentrenamiento continuo.

La combinación de **Python, PySpark, SageMaker, Airflow, GitHub Actions** y demás servicios de **AWS**, proporciona un flujo de MLOps moderno y profesional que automatiza las tareas críticas del proceso. Esto reduce errores, acelera la iteración de modelos y garantiza reproducibilidad en cada etapa. El uso de contenedores Docker, junto con un enfoque híbrido de despliegue (cloud + local mediante ONNX), permite que el sistema se adapte a distintos entornos hospitalarios, incluso en escenarios de baja conectividad o políticas de red restrictivas.

Además, el pipeline está preparado para manejar los desafíos particulares del dominio clínico: datos heterogéneos (EHR, imágenes, texto), desbalance de clases, enfermedades huérfanas con pocas muestras y la necesidad de explicabilidad basada en SHAP/LIME, lo cual es indispensable para generar confianza en los profesionales de la salud.

En conjunto, el pipeline presentado constituye una hoja de ruta técnica y operativa para implementar sistemas de IA clínica confiables, auditables y escalables, capaces de apoyar la toma de decisiones médicas en escenarios reales, con un enfoque integral de MLOps que abarca tanto los retos técnicos como los requisitos éticos, regulatorios y operativos del sector salud.